CNN

核心思想：

1. Local Connectivity（局部连接）

   普通神经网络是所有像素到神经元，导致参数巨大。

   CNN 是局部像素到一个神经元。

2. Parameter Sharing（参数共享）

   Shift Invariance Assumption: If a feature is useful at spatial position (x, y), then it should also be useful for other positions (x', y')

卷积理解：

$$(f * g)(t) = \int f(\tau)g(t-\tau)d\tau$$

不同 t，相当于向右移动函数 $g(-x)$ t 个单位，每个位置计算重叠部分乘积，然后求和。

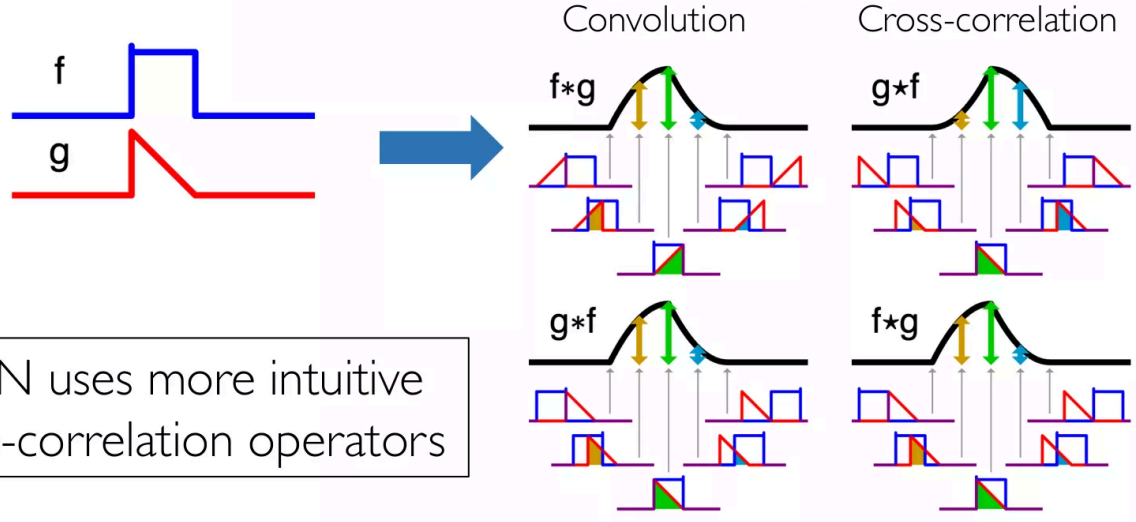
然后 CNN 就相当于一个二维卷积。尽管 CNN 是没有函数翻转，在数学中叫做 cross-correlation（互相关）。

注意卷积是可交换的，而 CNN 用到的 cross-correlation 是不可交换的。

- **Cross-correlation** is a similarity measure of two series $f(t), g(t)$ as a function of the lag (displacement) of one relative to the other.

$$(f \star g)(n) = \sum_{m=-\infty}^{\infty} \overline{f(m)} g(m+n)$$

$$[f(t) \star g(t)](t) = \left[\overline{f(-t)} * g(t)\right](t)$$



从这张图也可以看出，convolution 是翻转后者后，将其向右移动。cross-correlation 是不翻转后者，将其向左移动。转换关系：

$$\boxed{[f(t) \star g(t)](t) = [\overline{f(-t)} * g(t)](t)}$$

CNN filter 在空间上很小（local receptive field），但在 channel 方向必须穿透整个深度。

所以：

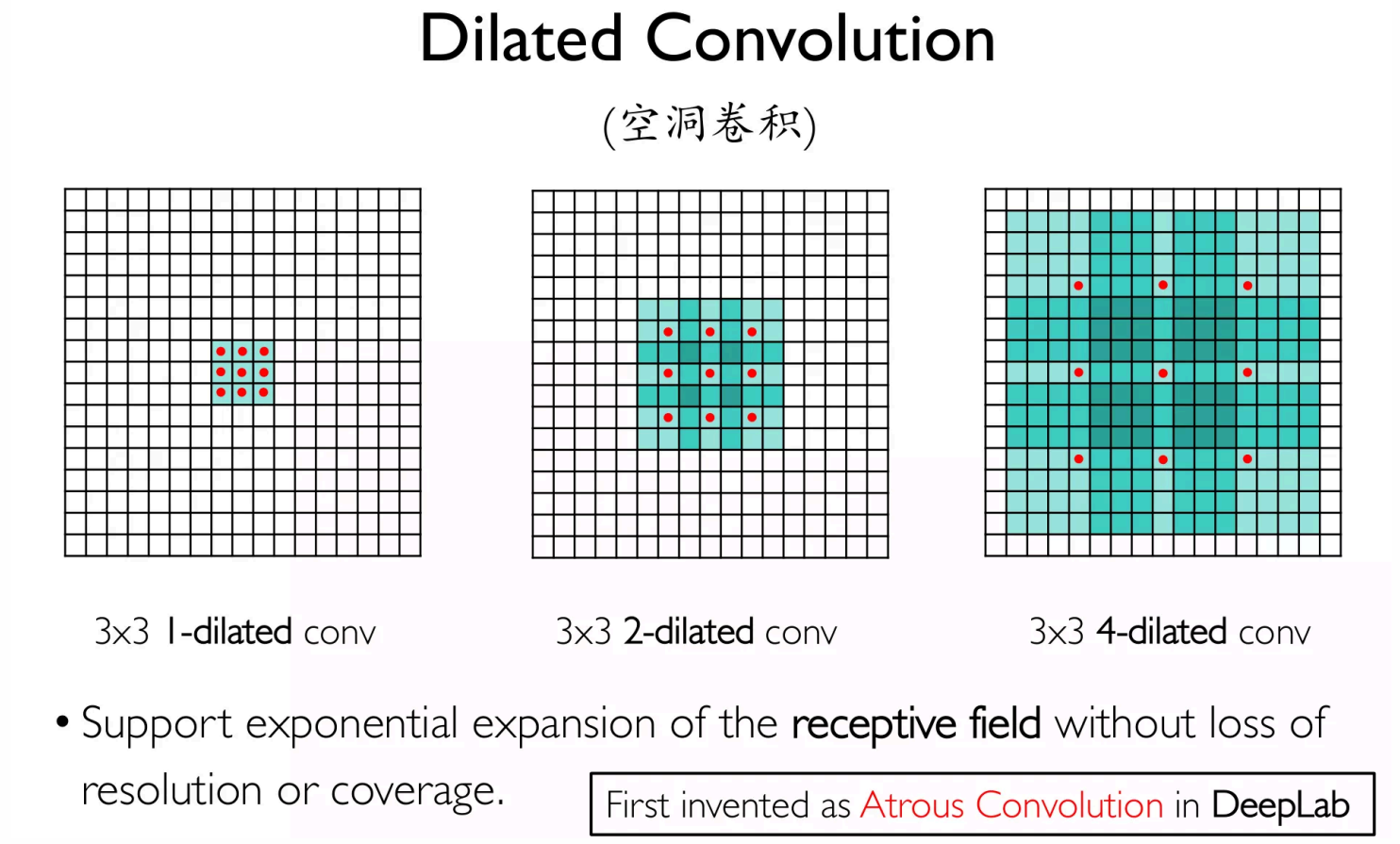
$$\boxed{5\times5 \text{ filter on RGB image} = 5 \times 5 \times 3}$$

增大感受野（receptive field）：

1. 增大 kernel
2. 堆多层卷积

但两者都会带来计算量增加。

Dilated Convolution（空洞卷积 / Atrous Convolution）：

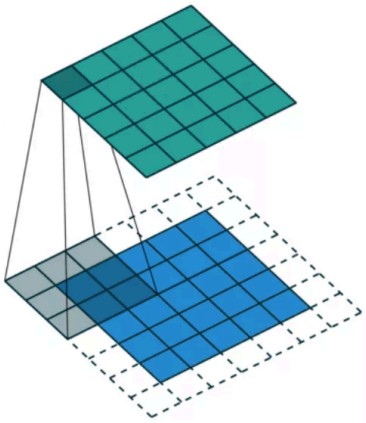


通过指数选择，由唯一分解定理，理论上可以表示完整信息。

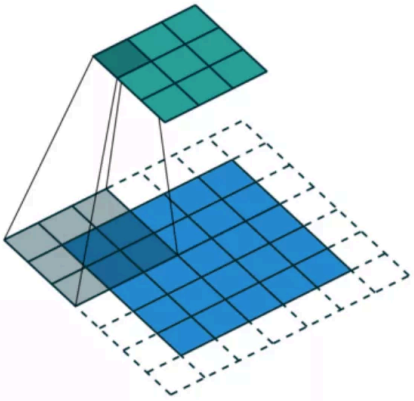
重要参数：

1. Kernel Size
2. Stride（步幅）：filter 每次移动多少格
3. Padding（填充）：输入边缘补多少圈
4. Dilation（膨胀率/空洞率）：filter 内部元素之间隔多远

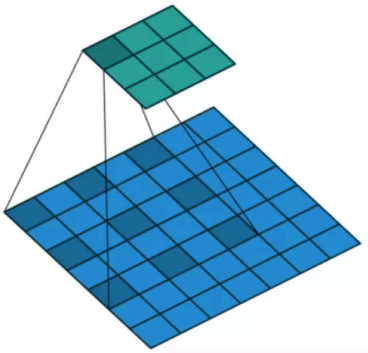
Convolution Operators



Convolution
Kernel Size: 3x3
Stride: 1
Padding: 1
Dilation: 1



Stride Convolution
Kernel Size: 3x3
Stride: 2
Padding: 1
Dilation: 1

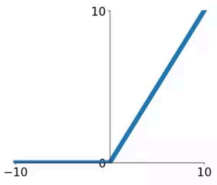


Dilated Convolution
Kernel Size: 3x3
Stride: 1
Padding: 0
Dilation: 2

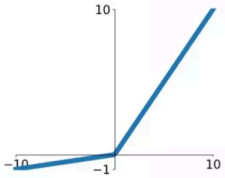
引入激活函数：

Activation Functions

ReLU
$\max(0, x)$



Leaky ReLU
$\max(ax, x)$



Advantages:

- Does not saturate (in positive region)
- Very computationally efficient
- Converges much faster than sigmoid/tanh in practice (e.g. 6x)
- Actually more biologically plausible than sigmoid

Disadvantages:

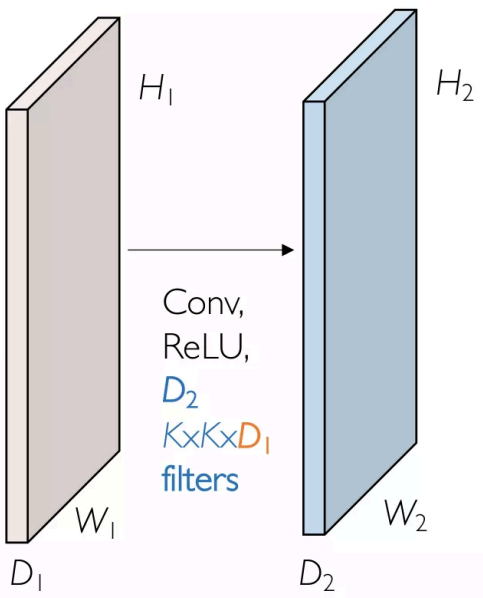
- **Not zero-centered output**
- **An annoyance → Leaky ReLU**

CNN is a stack of Convolutional Layers, interspersed with activation functions. Note: Shrinking too fast in the spatial dimensions is not good, doesn't work well.

超参数总结：

Convolutional Layer: Hyperparameters

- Accepts an input volume of size $W_1*H_1*D_1$
- Requires four hyper-parameters
  - Number of filters: D_2 (depth of next layer)
  - Receptive field (kernel size): K
  - Stride: S
  - The amount of zero Padding: P
- Produce an output volume of size $W_2*H_2*D_2$
  - $W_2 = \text{floor}[(W_1 - K + 2P) / S] + 1$
  - $H_2 = \text{floor}[(H_1 - K + 2P) / S] + 1$
- With parameter sharing, it requires $(K*K*D_1 + 1)*D_2$ parameters



卷积负责“找特征”，Pooling 负责“压缩特征，同时降低对位置的敏感性”。

CNN经典结构：Conv -> ReLU -> Pooling

Max Pooling 取最大。Average Pooling 取平均。实际 max pooling 更常见，因为特征检测通常更关注有没有出现。

池化：

- Alleviate excessive sensitivity of convolutional layers to locations
- Reduce the resolution of images through the processing pipeline

Operates over each feature map independently: Pooling reduces dimensions of width and height. Depth keeps unchanged before and after pooling

空间金字塔池化（Spatial Pyramid Pooling, SPP）

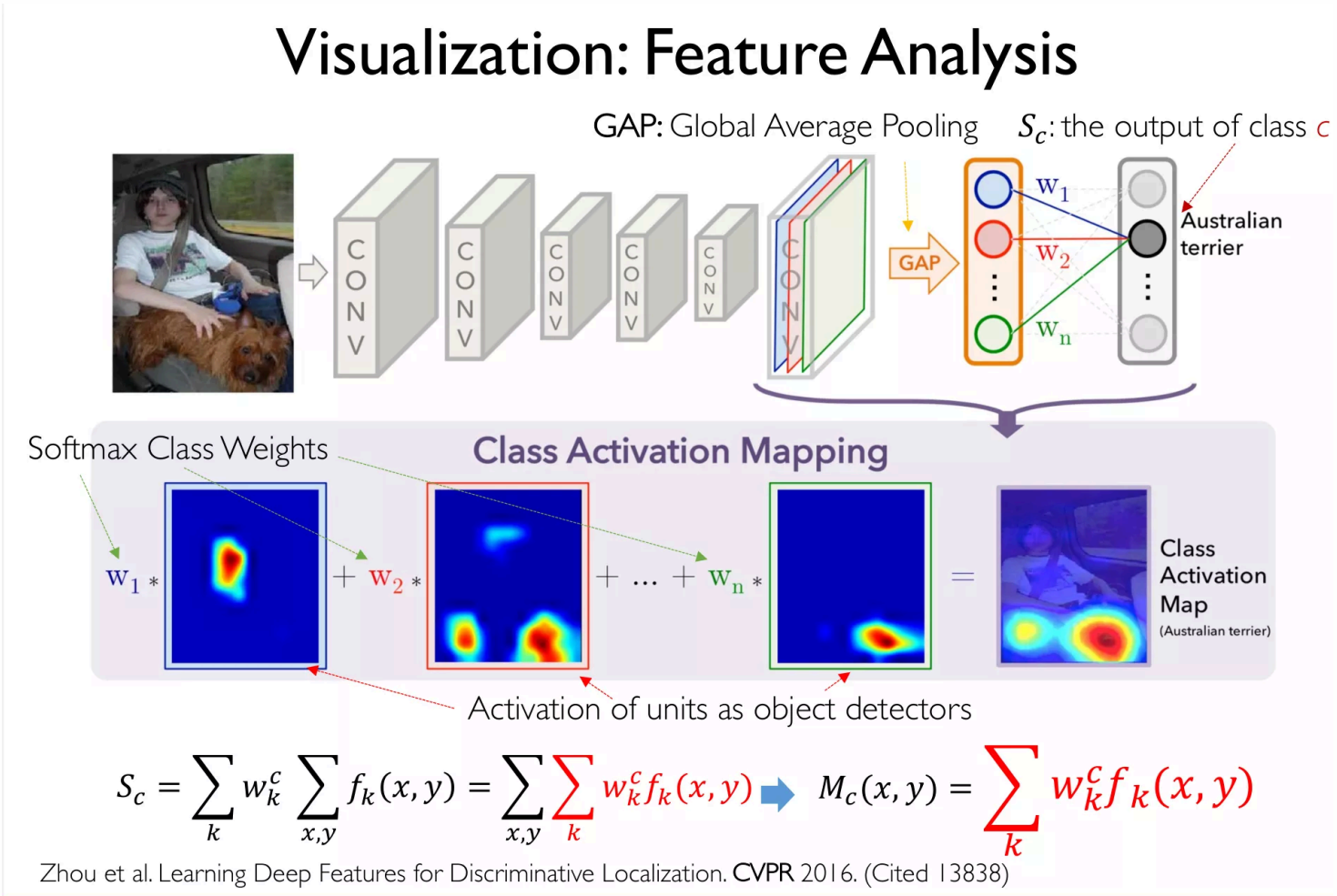
由于 CNN 后面一般会接全连接层（用于最终处理获得想要的），所以传统 CNN 一般要求输入图片大小固定。而 SSP 就是通过池化，把它变成固定长度，从而不限制输入图片大小。

SSP 其实就是通过固定想要输出的规模大小，而自适应 Pooling 参数。

LeNet 和 AlexNet 对比：

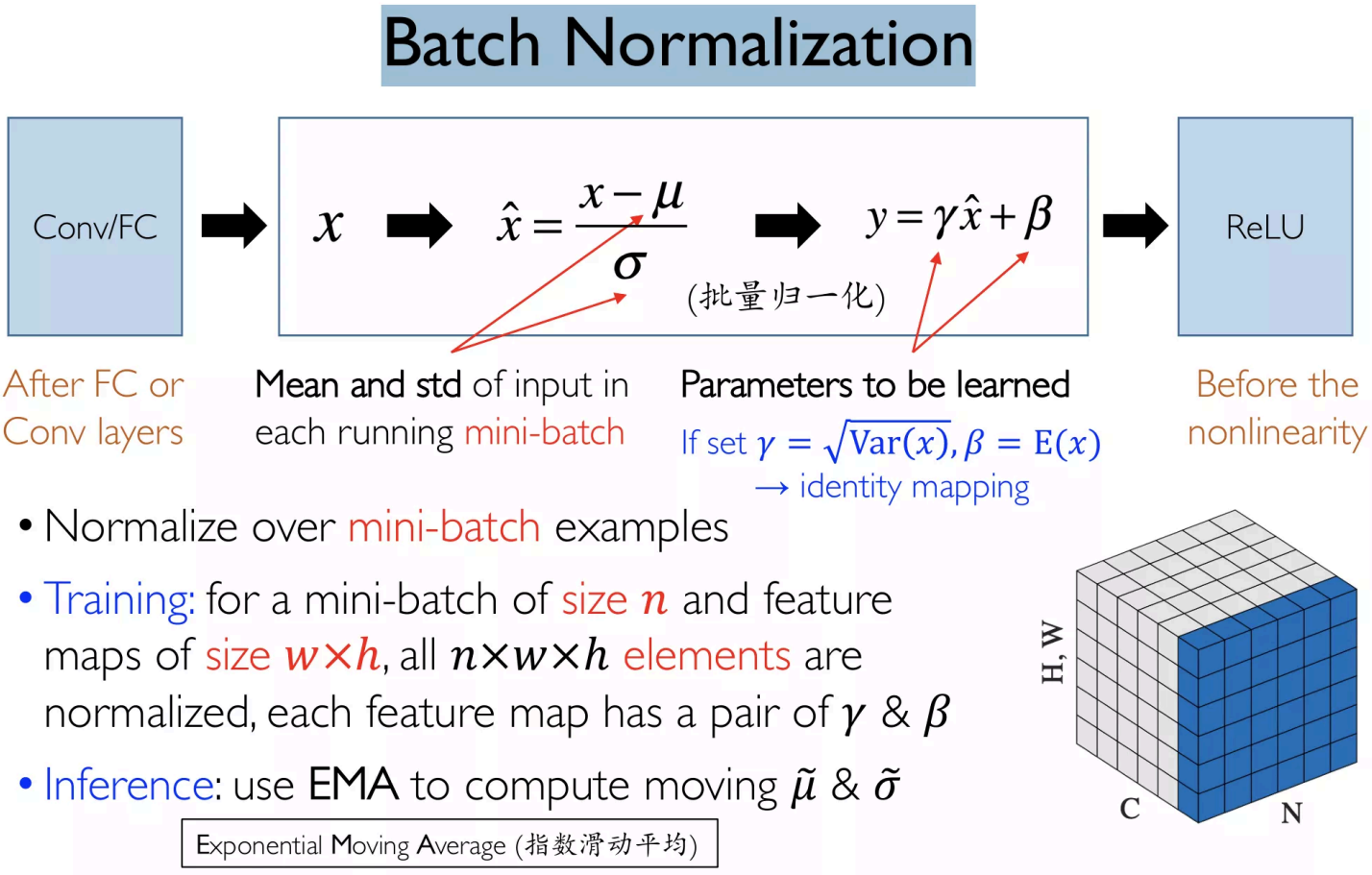
	LeNet	AlexNet
年份	1998	2012
任务	手写数字	ImageNet
图片	32×32灰度	大尺寸RGB
层数	浅	深
参数	几十万	6000万
激活	sigmoid/tanh	ReLU
训练	CPU	GPU
意义	证明CNN可行	推动深度学习革命

Class Activation Mapping, CAM:



CNN 最后一层已经知道“哪些 feature 能判断类别”，CAM 利用这些 feature 对类别的权重，把 feature map 加权组合回来，得到一张热力图，显示图片中哪些区域最影响分类结果。因为用的是 GAP 之前的 feature map，所以可以插值还原到原图上。

Batch Normalization



- Normalize over mini-batch examples
- Training: for a mini-batch of size n and feature maps of size $w \times h$, all $n \times w \times h$ elements are normalized, each feature map has a pair of γ & β
- Inference: use EMA to compute moving $\tilde{\mu}$ & $\tilde{\sigma}$

Exponential Moving Average (指数滑动平均)

Sergey Ioffe and Christian Szegedy. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. ICML, 2015.

重点是引入了 $gamma$ 和 $beta$，防止表达能力受限。

BN 不会限制网络，因为它有能力恢复原来的数据（取特定 $gmma$ 和 $beta$ 时）。

放在 ReLu 前是为了防止输入分布偏移导致神经元坏死。

矩阵分解。

在相同 learning rate 下，BN 和非 BN 的测试准确率可以比较；而更大的 learning rate 对 BN 网络更加友好，非 BN 网络可能直接发散。BN 让网络对 learning rate 没那么脆弱，可以更大胆地进行梯度下降。

Training

每个 mini-batch：

$$\mu, \sigma \leftarrow \text{当前 batch}$$

然后用它进行 normalization。

同时不断更新：

$$\mu_{\text{running}}, \sigma_{\text{running}}$$

Inference

不再计算当前 batch 的统计量，而是：

$$\mu, \sigma \leftarrow \mu_{\text{running}}, \sigma_{\text{running}}$$

然后：

$$x \rightarrow \frac{x - \mu_{\text{running}}}{\sigma_{\text{running}}} \rightarrow \gamma x + \beta$$

推理阶段的 μ 和 σ 是由训练阶段通过 EMA（Exponential Moving Average，指数滑动平均）得到的，这也隐含了早期模型不稳定的时候占据的权重是指数级别，只占很少一部分。

由于 BN 在训练时带来的影响仍然是线性的，所以其实并没有增强模型的表达能力。只是有利于训练。

BN 有效性解释：

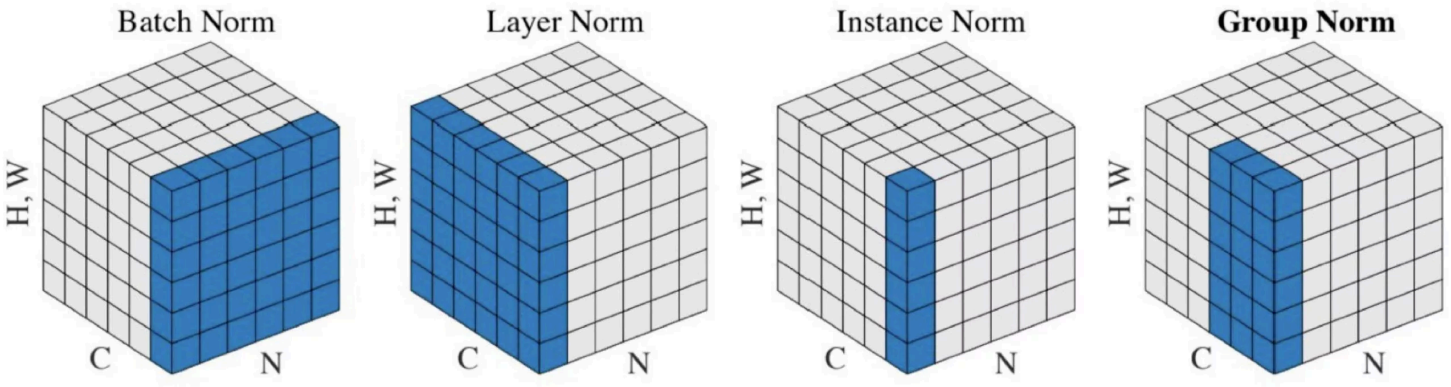
- ICS 解释：BN 让每一层看到的输入更稳定，因此更容易训练。被 **Noisy BN** 实验推翻。
- Smoothness 解释：BN 改善了优化时 loss 和梯度的变化性质，因此更容易训练。

A small batch leads to inaccurate estimation of the batch statistics. 因为 BN 希望根据大数定律，将每个样本视为独立。

GN: 不跨样本计算，而是在每张图片内部，把 Channel 分组进行归一化。

方法	统计范围	是否跨 Batch
BN	同一 Channel 的 N,H,W	是
LN	同一样本的 C,H,W	否
IN	同一样本、同一 Channel 的 H,W	否
GN	同一样本、同一组 Channel 的 C/G,H,W	否

GN 是 LN 子集。是 IN 的超集。



BN

BN 训练时通过 mini-batch 估计每个 Channel 的统计量。

标准 BN 推理时使用固定的 running statistics，使得：

- 训练积累的统计尺度能够延续到推理阶段。
- 同一张图片不会因为和哪些其他图片组成 Batch 而改变预测。
- 即使 Feature Map 的空间尺寸很小，也不需要重新估计统计量。

GN

GN 从设计上就是针对单个样本内部的一组 Channel 进行归一化。
因此，无论训练还是推理，都是同一个公式、同一个统计范围。
它并不需要记录训练数据的总体均值和方差。

不变性 Invariance：比如一般的图像分类，把网球稍微移到右边，输出仍然是 Tennis ball。

$$f(T(x)) = f(x)$$

等变性 Equivariance：比如目标定位，把网球移到右边，输出坐标也应该相应向右移动。

$$f(T(x)) = T(f(x))$$

CNN is designed to be invariant to changes by adding pooling layer.

卷积天然具有平移等变性，Pooling 把一部分精确的位置差异丢掉了，因此模型不再那么在意网球具体位于窗口中的哪个位置。

在真实图像数据集上训练的 CNN，可能过度依赖纹理（texture），而不是充分利用物体的整体形状（shape）。解决方案：Augmentation by Stylization（通过风格化进行数据增广）。即保留原图的主要内容和形状，但改变它的纹理、颜色风格和局部外观。

CNN 可能过度依赖物体周围的环境和上下文。解决方案：把不同图片中的物体和场景进行各种组合，故意制造一些不寻常的图像。

Max Pooling 代价：模型保留了“这个区域存在强响应”的信息，却丢失了一部分精确的位置关系。所以就引入了 Capsule Network（胶囊网络）：它尝试在识别物体的同时，更好地保留姿态和空间关系。

胶囊网络：

对比维度	传统 CNN	胶囊网络
单个基本单元的输出	通常是标量特征	向量形式的胶囊
实体存在程度	由激活值或分类输出表达	由胶囊向量长度表达
姿态信息	由特征图和网络隐式编码	尝试在胶囊向量中显式编码
层间信息传递	常规固定连接	通过动态路由调整连接权重
空间信息	Pooling 可能丢弃部分位置细节	试图保留实体及其姿态关系
额外训练目标	取决于具体网络	原论文使用重建损失辅助训练

没有看懂

预训练与后训练

1. Big Transfer（BiT）: Training on larger datasets is vital for better transferability.
2. Domain Adaptive Transfer（DAT）: Uses importance weighting to carefully choose the pre-training data that are most relevant to the target task.

Supervised pre-training gains high-level semantic knowledge.

Generative pre-training gains low-level representations.

Contrastive pre-training gains mid-level & low-level representations.

Instance Discrimination: 增光作为正样本，其他作为负样本

Non-parametric Softmax：普通的神经网络分类器通常会在最后一层学习一组分类权重，而 Non-parametric Softmax 是将 Memory Bank 中保存的各个实例特征，作为比较对象。

普通 Softmax 分类器

Encoder 特征 h

↓

可训练权重 W

↓

各类别概率

分类权重通过梯度下降学习。

Non-parametric Softmax

当前图片特征 v

↓

与 Memory Bank 中的 $v_1, \ldots, v_n$ 比较

↓

各实例概率

比较对象来自记忆库中保存的实例特征，而不是一个独立学习的分类权重矩阵。

对比学习基本任务：

假设有一张图片 x，我们随机增广两次，得到两个视图：

$$x^q = T_q(x), \qquad x^k = T_k(x)$$

它们来自同一张原图，因此是正样本对。
我们把两个视图送进网络，得到：

$$q = f_q(x^q), \qquad k^+ = f_k(x^k)$$

- q：Query（查询向量）。
- k^+：正样本 Key（键向量）。
  我们希望 q 和 k^+ 相似。

同时，还需要大量负样本 $k_1^-, k_2^-, \ldots$，让 q 能够区分不同的图片。

直接使用当前 mini-batch 会导致负样本很少，所以就有了 MoCo：把负样本特征存储在一个队列中，而不是每次都只从当前 mini-batch 获取。

简单流程：

MoCo 的一次训练迭代

1. **生成两个视图**

   同一张图片经过两次随机增广，产生 x^q 和 x^k。

2. **计算 Query**

   Query Encoder 处理 x^q，得到：

$$q = f_q(x^q)$$

3. **计算正样本 Key**

   Momentum Encoder 处理 x^k，得到：

$$k^+ = f_k(x^k)$$

   这个 Key 与当前 Query 对应同一张原图，因此是正样本。

4. **取出负样本**

   从队列中读取已经保存的 Key：

$$k_1^-, k_2^-, \ldots, k_K^-$$

   它们作为当前 Query 的负样本候选。

5. **计算对比损失**

   让 Query 更接近当前正样本 Key，同时区分队列中的负样本。

6. **更新 Query Encoder**

对比损失反向传播，通过优化器更新 θ_q。

7. 更新 Momentum Encoder

使用动量公式更新 θ_k，而不是直接对它做梯度下降。

8. 更新队列

将当前 Batch 生成的 Key 加入队列，并移除最旧的 Key，保持队列大小固定。

↓

对比维度	普通对比学习（经典方法）	SimCLR	SimSiam
核心思想	拉近正样本，区分负样本	同样的对比学习思想，简化训练框架	让两个视图的表示互相预测，不需要负样本
正样本	通常来自同一图片的不同增广视图	同左	同左
负样本	需要	需要，直接使用当前 Batch 中其他视图	**不需要**
损失函数	典型为 InfoNCE 或其他对比损失	NT-Xent（InfoNCE 的一种形式）	负余弦相似度，结合 Stop-gradient
Encoder	依具体方法而定	两个视图共享同一套 Encoder 参数	两个分支共享同一套 Encoder 参数
投影头 Projection Head	依具体方法而定	有，使用非线性 MLP	有，另加 Predictor 预测头
特征队列	某些方法需要，例如 MoCo	不需要	不需要
动量编码器	某些方法需要，例如 MoCo	不需要	不需要
对 Batch Size 的要求	取决于具体方法	原始 SimCLR 强调大 Batch	不依赖大量负样本，因此不需要大 Batch
防止表示坍塌的主要机制	负样本提供区分约束	通过正负样本对比	通过 Predictor、Stop-gradient 等结构设计
训练资源特点	取决于实现	大 Batch，计算与显存开销较大	不需要维护大量负样本，通常更节省这方面的资源
下游迁移	使用预训练 Encoder	通常保留 Encoder，丢弃投影头	通常保留 Encoder，丢弃投影头和 Predictor